

CONNECTIONS BETWEEN SCHEDULE-FREE OPTIMIZERS, ADE-MAMix, AND ACCELERATED SGD VARIANTS

Anonymous authors
Anonymous submission

ABSTRACT

Recent advancements in deep learning optimization have introduced new algorithms, such as Schedule-Free optimizers, AdEMAMix, MARS and Lion which modify traditional momentum mechanisms. In a separate line of work, theoretical acceleration of stochastic gradient descent (SGD) in the noise-dominated regime has been achieved by decoupling the momentum coefficient from the current gradient’s weight. In this paper, we establish explicit connections between these two lines of work, and we test what these connections predict empirically. Under a controlled comparison of seven optimizers on a 150m-parameter language modeling task, we find a clear and consistent ordering: AdEMAMix and Simplified-AdEMAMix, the two methods that most closely match accelerated SGD, attain the best validation loss, while Schedule-Free AdamW tracks its accelerated-SGD counterpart almost exactly, exactly as the theory predicts. The advantage of the accelerated methods is largest in the small-batch (noise-dominated) regime and, as the theory anticipates, narrows at large batch size relative to Adam. Building on these insights, we introduce a modification to AdEMAMix, termed Simplified-AdEMAMix, which matches AdEMAMix’s performance across both the large- and small-batch settings while eliminating the need for two separate momentum terms, and thus the associated memory overhead. The code for Simplified-AdEMAMix will be released upon acceptance.

1 INTRODUCTION

Recently, numerous optimization algorithms have been introduced for deep learning such as Lion (Chen et al., 2023), ScheduleFreeSGD/AdamW (Defazio et al., 2024), and AdEMAMix (Pagliardini et al., 2024). While these optimizers have been proposed with distinct motivations, they share a common characteristic: each modifies the momentum scheme employed in optimization.

A separate body of theoretical research has focused on accelerating gradient descent in noisy environments. Although classical momentum methods, such as heavy-ball or Nesterov momentum, are sufficient to accelerate deterministic gradient descent (particularly for quadratic functions), they do not accelerate SGD (Jain et al., 2018; Liu & Belkin, 2020). This limitation has led to the development of alternative momentum schemes aimed at achieving acceleration in the presence of noise (Jain et al., 2018; Vaswani et al., 2019; Liu & Belkin, 2020; Gupta et al., 2023). Notably, all proposed accelerated SGD methods can be interpreted as decoupling the momentum coefficient from the weight assigned to the current gradient in the optimizer update.

Our primary contribution is to establish a direct connection between the ideas developed in these two research directions, and then to show that this connection has measurable predictive value. Specifically, we demonstrate that Schedule-Free SGD is mathematically equivalent to performing accelerated SGD followed by weight averaging. Furthermore, optimizers such as Lion, Schedule-Free AdamW, and AdEMAMix can be understood as combining preconditioning techniques with accelerated SGD approaches. While certain aspects of these connections have been noted in prior literature (Defazio, 2021), to the best of our knowledge, the relationship between these recently proposed optimizers and accelerated SGD has not been formally established before. The value of making it explicit is that it yields concrete, testable predictions about which optimizer should win under which batch-size regime—predictions we confirm empirically below.

To test what these theoretical findings predict, we run a controlled comparison on a 150m decoder-only transformer model, trained on 15b tokens with a small batch size of 32k tokens, a setting deliberately chosen to place training firmly in the noise-dominated regime where accelerated SGD is expected to help. The measured outcome matches the

prediction on two counts. First, Schedule-Free AdamW and accelerated SGD-based AdamW (Algorithm 3)—which our analysis shows to be equivalent up to weight averaging—land on essentially the same validation loss. Second, across the full set of methods the accelerated variants come out ahead in this small-batch regime, with AdEMAMix and Simplified-AdEMAMix leading the field. Crucially, this advantage is regime-dependent rather than universal: at sufficiently large batch sizes the gap over Adam closes, precisely the behavior the connection to accelerated SGD anticipates.

Our main contributions are stated below:

1. We establish precise theoretical connections between accelerated SGD and recently proposed optimizers, such as Schedule-Free SGD and AdEMAMix.
2. We provide empirical validation through a head-to-head comparison of seven optimizers on a 150-million-parameter decoder-only transformer—AdamW (with cosine decay and with weight averaging), Schedule-Free AdamW, accelerated-SGD-based AdamW, MARS, AdEMAMix, and Simplified-AdEMAMix. In the small-batch regime the ranking is unambiguous: AdEMAMix and Simplified-AdEMAMix, the methods that align most closely with accelerated SGD, achieve the lowest validation loss, and Schedule-Free AdamW matches its accelerated-SGD counterpart as the equivalence predicts.
3. We show that this ranking is regime-specific rather than absolute: as anticipated from the equivalence to accelerated SGD, the advantage over Adam vanishes at large batch sizes. Notably, in that regime Adam with momentum scheduling recovers the full performance of AdEMAMix.
4. In the large-batch regime we further find a clear and reproducible gap in the other direction: Schedule-Free AdamW trails AdamW with cosine decay by a substantial margin, which we trace to the intrinsic coupling of momentum and weight-averaging coefficients in Schedule-Free optimizers.
5. We introduce a modification to AdEMAMix, termed Simplified-AdEMAMix, which matches AdEMAMix in both the large- and small-batch regimes while using only a single momentum term, thereby removing the extra memory overhead.

2 RELATED WORK

We review the existing literature on accelerated SGD variants and on the optimization algorithms we later place in direct comparison.

Jain et al. (2018) introduced an accelerated SGD variant that demonstrated improved convergence rates for the least-squares problem. Kidambi et al. (2018) further simplified the update rule for this variant and formally established that momentum does not provide acceleration in this specific case. Subsequent works (Liu & Belkin, 2020; Vaswani et al., 2019; Gupta et al., 2023) extended these results to general convex and strongly convex functions under various theoretical assumptions.

A recurring theme across practical optimizers is that they reintroduce, under different names, the same decoupling that defines accelerated SGD. Lucas et al. (2019) combine a weighted sum of multiple momentum terms, each with its own coefficient, into the final update. Ma & Yarats (2019) build an optimizer explicitly on the theoretical framework of Jain et al. (2018). More recently, Chen et al. (2023) obtained an optimizer through genetic search that, like earlier accelerated SGD variants, assigns different weights to the gradient and to the momentum term. AdEMAMix (Pagliarini et al., 2024) likewise blends two distinct momentum scales in its update. The comparison in this paper places representatives of these families side by side on a single benchmark, so that the structural resemblances can be read off as differences in measured performance.

3 BACKGROUND

3.1 MOMENTUM

Momentum is a well-established technique for accelerating the convergence of gradient descent in deterministic settings. The momentum update for weights w_t , with a momentum coefficient β , is given by:

$$m_t = \beta m_{t-1} + \nabla f(w_t); \quad w_t = w_{t-1} - \eta m_t$$

3.2 WEIGHT AVERAGING

Weight averaging is a widely used technique in stochastic optimization to reduce noise in the iterates. Instead of returning the final iterate w_T , a weighted average $\bar{w}_T$ of the iterates is computed, where the weights are denoted by γ_t :

$$\bar{w}_T = (1 - \gamma_T)\bar{w}_{T-1} + \gamma_T w_T$$

All instances of weight averaging in this paper utilize coefficients γ_t of the form:

$$\gamma_t \approx 1 - \frac{1}{\delta t}$$

for some constant $0 \leq \delta \leq 1$.

3.3 ACCELERATED SGD

In this section, we provide a generalized framework encompassing many accelerated SGD methods:

$$m_t = \beta_{a,t} m_{t-1} + g_t, \quad w_{t+1} = w_t - \eta_{a,t} m_t - \alpha_{a,t} g_t \quad (1)$$

where $\beta_{a,t}, \alpha_{a,t}, \eta_{a,t}$ are (possibly time-dependent) scalar coefficients, and g_t represents the stochastic gradient evaluated at w_t . We use the subscript ‘a’ to indicate coefficients that adhere to this specific accelerated SGD formulation.

Two facts anchor this framework as the right yardstick for the comparisons that follow. Setting $\alpha_{a,t} = 0$ recovers standard SGD with momentum, so the single coefficient $\alpha_{a,t}$ cleanly separates ”accelerated” from ”not accelerated.” And the framework is broad: many accelerated SGD algorithms in the literature—such as those of Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023)—fall directly within it, with a precise demonstration of this equivalence given in Appendix B.

4 CONNECTIONS BETWEEN EXISTING OPTIMIZERS AND ACCELERATED SGD

In this section, we theoretically establish precise connections between existing optimizers, such as Schedule-Free optimizers and AdEMAMix, and accelerated SGD. These connections are what let us later read Figure 1 as more than a leaderboard: each rung of the empirical ranking corresponds to a structural feature identified here. Based on the same insights, we propose a simplified variant of AdEMAMix that uses a single momentum term yet, as the experiments confirm, matches AdEMAMix across both the small- and large-batch regimes.

4.1 SCHEDULE-FREE SGD

Schedule-Free SGD (Defazio et al., 2024) is a recently introduced constant learning rate optimizer designed to eliminate the need for scheduling. Following the notation used in Defazio et al. (2024), the update equations are given by:

$$\begin{aligned} y_t &= (1 - \beta)z_t + \beta x_t \\ z_{t+1} &= z_t - \gamma g(y_t) \\ x_{t+1} &= (1 - c_{t+1})x_t + c_{t+1}z_{t+1} \end{aligned}$$

Here, y_t represents the current model weights (where the gradient is evaluated), while x_t denotes the weights used for evaluation.

We first express the update in terms of y_t and m_t , where we define:

$$m_{t+1} = \frac{x_t - z_{t+1}}{\gamma}. \quad (2)$$

Further simplifying m_{t+1} , we obtain:

$$m_t = \frac{x_t - z_{t+1}}{\gamma} \quad (3)$$

$$= \frac{x_t + \gamma g_t - z_t}{\gamma} \quad (4)$$

$$= \frac{x_t - z_t + \gamma g_t}{\gamma} \quad (5)$$

$$= \frac{(1 - c_t)(x_{t-1} - z_t) + \gamma g_t}{\gamma} \quad (6)$$

$$= (1 - c_t)m_{t-1} + g_t. \quad (7)$$

Thus, m_t follows the momentum update in Equation (1) with $\beta_{a,t} = 1 - c_t$. Given m_t , we now examine the update for y_t :

$$y_{t+1} = (1 - \beta)z_{t+1} + \beta x_{t+1} \quad (8)$$

$$= (1 - \beta)(z_t - \gamma g_t) + \beta((1 - c_{t+1})x_t + c_{t+1}z_{t+1}) \quad (9)$$

$$= (1 - \beta)z_t + \beta x_t - (1 - \beta)\gamma g_t + \beta c_{t+1}(z_{t+1} - x_t) \quad (10)$$

$$= y_t - \gamma[\beta c_{t+1}m_t + (1 - \beta)g_t]. \quad (11)$$

Thus, y_t follows the weight update in Equation (1) with $\eta_{a,t} = \gamma\beta c_{t+1}$ and $\alpha_{a,t} = \gamma(1 - \beta)$, where $w_t = y_t$. Consequently, y_t in Schedule-Free SGD precisely follows the accelerated SGD framework. However, x_t is used for evaluation in Schedule-Free SGD. We now analyze the dynamics of x_t :

$$x_{t+1} = (1 - c_{t+1})x_t + c_{t+1}z_{t+1}$$

$$x_{t+1} = (1 - c_{t+1})x_t + c_2 \left(\frac{y_{t+1} - \beta x_{t+1}}{1 - \beta} \right)$$

$$x_{t+1}(1 - \beta + c_{t+1}\beta) = (1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1}$$

$$x_{t+1} = \frac{(1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1}}{(1 - c_{t+1})(1 - \beta) + c_{t+1}}.$$

Thus, x_t is a weighted average of y_t . Recursively expanding x_t confirms that it is an exponential average of y_t when c_t is a constant. This establishes that Schedule-Free SGD can be understood as accelerated SGD followed by weight averaging.

The decomposition above pinpoints two distinct sources of Schedule-Free SGD's benefit, which our experiments later disentangle:

1. A performance gain over standard SGD with momentum, inherited directly from its equivalence to accelerated SGD—this is the component that places it above plain AdamW in the small-batch comparison of Figure 1.
2. The ability to run at a constant learning rate with no schedule, enabled by weight averaging (specifically, tailed weight averaging; see Section 4.1.3).

We note two advantages unique to Schedule-Free SGD/Adam:

- It does not require additional memory for weight averaging.
- It eliminates the need for an explicit weight averaging coefficient as a hyperparameter.

This same coupling, however, carries a cost that only surfaces at scale: in Section 5.1 we show that it is precisely this fixed link between the momentum and weight-averaging coefficients that leaves Schedule-Free AdamW trailing AdamW with cosine decay by a clear margin in the large-batch regime.

4.1.1 CASE: $\beta = 0.0$

As noted in (Defazio et al., 2024), when $\beta = 0$, Schedule-Free SGD reduces to standard SGD with weight averaging. Since $c_t = 1/t$, it applies weight averaging from the beginning.

4.1.2 CASE: $\beta = 1.0$

As noted in (Defazio et al., 2024), when $\beta = 1$, Schedule-Free SGD reduces to standard momentum SGD, with the momentum coefficient $\beta_{a,t}$ scaling as $1 - 1/t$.

4.1.3 CASE: $\beta = 0.9$

For $\beta = 0.9$, the default setting in Schedule-Free SGD:

- As c_t scales as $1/t$, momentum grows as $1 - 1/t$.
- The ratio of the weight assigned to the current gradient versus momentum is fixed at $(1 - \beta)/(\beta c_{t+1}) \approx 0.11$.
- Weight averaging is applied approximately over the most recent 10% of the iterates.

4.2 LION

The update rule for Lion (Chen et al., 2023) is given by:

$$m'_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\theta_t = \theta_{t-1} - \eta \text{sign}(m'_t)$$

$$m_t = \beta_2 m_{t-1} + (1 - \beta_2) g_t.$$

Lion (Chen et al., 2023) can be directly interpreted as an accelerated SGD method followed by a coordinate-wise sign operation.

4.3 MARS

In this section, we demonstrate that the practical version of the recently proposed optimizer MARS (Yuan et al., 2024), referred to as MARS-Approx, follows the accelerated SGD framework, supplemented by a preconditioning step. The update equations (ignoring bias correction and clipping) are given by:

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ m_t &= \beta_1 m_{t-1} + (1 - \beta_1) c_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \\ x_{t+1} &= x_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

where m_t and v_t represent the first- and second-order momentum terms, respectively, and x_t denotes the model parameters. Rewriting the update using $\hat{m}_t = m_t - \gamma g_t$, we obtain:

$$\begin{aligned}
c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\
\hat{m}_t &= \beta_1 \hat{m}_{t-1} + (1 - \beta_1)(1 - \gamma)g_t \\
v_t &= \beta_2 v_{t-1} + (1 - \beta_2)c_t^2 \\
x_{t+1} &= x_t - \eta \frac{\hat{m}_t + \gamma g_t}{\sqrt{v_t} + \epsilon}
\end{aligned}$$

This formulation illustrates that the momentum update follows the general accelerated SGD framework, the one structural ingredient it shares with the leading methods. Where MARS differs is in its preconditioning, which is distinct from that of AdamW; we isolate the empirical consequence of this difference by including MARS directly in the small-batch comparison of Section 5.

4.4 ADEMAMIX

The recently proposed optimizer AdEMAMix (Pagliardini et al., 2024) shares structural similarities with accelerated SGD-based AdamW. However, instead of using a linear combination of the current gradient and the momentum term as in accelerated SGD, AdEMAMix maintains two distinct momentum terms with different coefficients and computes their linear combination. The algorithm is formally stated in Algorithm 1.

To simplify our analysis, we consider a variant of AdEMAMix with $\beta_1 = 0$. As demonstrated in Pagliardini et al. (2024), this simplified version matches the full version to within a negligible margin at small batch sizes, and our own small-batch experiments in Section 5 reproduce this near-parity. With $\beta_1 = 0$, AdEMAMix aligns exactly with the general accelerated SGD framework (Equation (1)). Furthermore, we show that the prescribed schedules for β_3 (momentum coefficient) and α (which controls the relative weight assigned to the current gradient) in AdEMAMix track, term for term, the theoretical schedules proposed for accelerated SGD (Gupta et al., 2023).

In smooth convex optimization, achieving acceleration in stochastic settings requires a momentum scheme of the form:

$$\beta_{a,t} = 1 - \frac{k}{t}$$

for some constant $k > 0$, as established by Gupta et al. (2023). The AdEMAMix optimizer approximately follows this scheme by scaling up β_3 accordingly.

Additionally, note that in accelerated SGD schemes, momentum is maintained in the standard form:

$$m_t = \beta_{a,t} m_{t-1} + g_t$$

whereas in Algorithm 1, the momentum update follows:

$$m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}.$$

For $\beta_{a,t}$ scaling as $1 - 1/t$, the accumulated contribution of past gradients in m_t in accelerated SGD grows proportionally to t . Similarly, the coefficient α in AdEMAMix also scales proportionally to t . This term-for-term correspondence is exactly what one would expect to translate into a performance edge, and it does: among all methods compared, AdEMAMix attains the best validation loss in the small-batch regime (Figure 1).

The same lens also predicts where AdEMAMix should stop helping. For large batch sizes, setting $\beta_1 = 0.0$ leads to a marked performance drop, as reported in Pagliardini et al. (2024) and reconfirmed in our large-batch experiments below. The reason is direct: Gupta et al. (2023) shows that the weight on the current gradient must decrease as batch size grows, yet AdEMAMix holds this weight fixed at 1—so the very mechanism that wins at small batch size becomes a liability at large batch size.

In the following section, we introduce a simplified variant of AdEMAMix that incorporates a weight on the current gradient, removes the need for scheduling α and maintains only a single momentum term. We then verify empirically

that this single-buffer variant loses nothing: it matches AdEMAMix in both the small- and large-batch setups, despite carrying half the momentum state.

Algorithm 1 Single step of AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters β_3, α . Warmup parameter T_{α, β_3} , note that we usually set it to T . β_{start} is usually set to β_1 .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_3^{(t)} \leftarrow f_{\beta_3}(t, \beta_3, \beta_{\text{start}}, T_{\alpha, \beta_3})$ and $\alpha^{(t)} \leftarrow f_{\alpha}(t, \alpha, T_{\alpha, \beta_3})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the fast EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + (1 - \beta_1) g^{(t)}$
 - 6: Update the slow EMA m_2 : $m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}$
 - 7: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 8: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{\hat{m}_1^{(t)} + \alpha^{(t)} \hat{m}_2^{(t)}}{\sqrt{\hat{\nu}^{(t)} + \epsilon}} \right)$
-

Algorithm 2 Single step of Simplified AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters $\alpha, \beta_{\text{start}}$. Warmup parameter T_{β_1} , note that we usually set it to T .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_1^{(t)} \leftarrow f_{\beta_1}(t, \beta_1, \beta_{\text{start}}, T_{\beta_1})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the EMA m_1 : $m_1^{(t)} \leftarrow \beta_1^{(t)} m_1^{(t-1)} + g^{(t)}$
 - 6: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 7: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{m_1^{(t)} + \alpha g^{(t)}}{\sqrt{\hat{\nu}^{(t)} + \epsilon}} \right)$
-

4.5 SIMPLIFIED ADEMAMIX

Building on the insights discussed above, we propose a simplified optimizer that eliminates the need for maintaining two separate momentum terms and removes the requirement for scheduling α . The optimizer is formally presented in Algorithm 2, where we employ theory-style momentum (instead of the exponential moving average (EMA) style). In the final update, we assign a fixed weight α to the gradient. We note that setting $\alpha = 0$ recovers the standard Adam optimizer (subject to appropriate transformations of η and β_1). The practical payoff is a strictly cheaper method that gives up no accuracy: in Section 5, Simplified-AdEMAMix lands on the same validation loss as AdEMAMix in both the small- and large-batch regimes, while storing only one momentum buffer instead of two.

5 EXPERIMENTS

We evaluate all methods on a single, common testbed so that differences in validation loss are attributable to the optimizer alone: a 150-million-parameter decoder-only transformer trained for language modeling on the C4 dataset, with a sequence length of 1024, a batch size of 32, and 15 billion tokens of training ($\approx 5 \times$ Chinchilla). The small batch size is chosen deliberately to keep training in the noise-dominated regime—the setting in which the accelerated-SGD connection predicts the accelerated methods should have the largest advantage.

Against this fixed backdrop we compare seven optimization algorithms, spanning standard baselines, their weight-averaged and accelerated counterparts, and the two AdEMAMix variants:

1. Standard AdamW with cosine decay

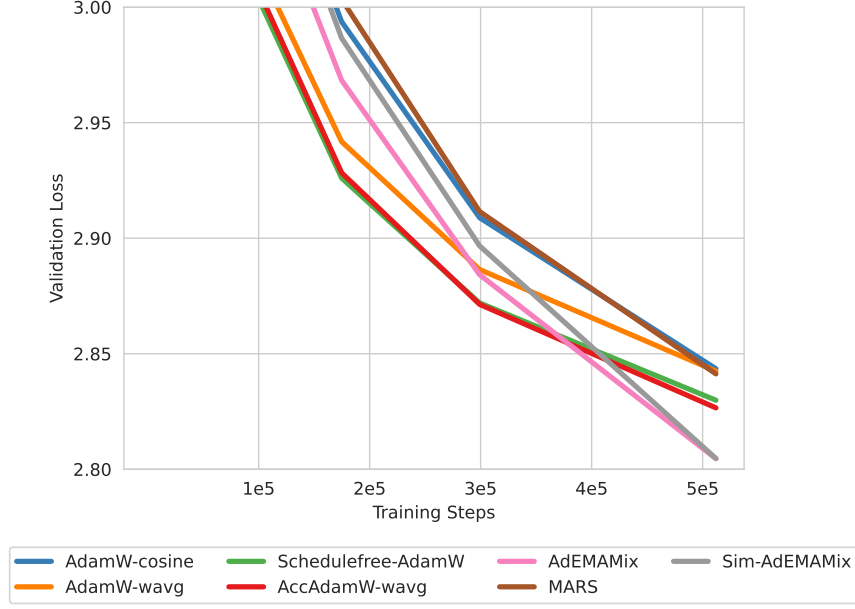


Figure 1: Small-batch (noise-dominated) regime: validation loss for the best run of each of the seven optimizers listed in Section 5, on the 150m decoder-only transformer language-modeling task. The ordering is consistent and clean—AdEMAMix and Simplified-AdEMAMix, the two methods that align most precisely with accelerated SGD variants, sit at the bottom of the loss curve (best), followed by the pair of accelerated/Schedule-Free AdamW methods, which in turn improve on plain AdamW with cosine decay. This is exactly the ranking the accelerated-SGD connection predicts for a noise-dominated setting.

Algorithm 3 Single step of accelerated SGD based Adam with weight averaging. For simplicity we ignore the initialization, other boundary effects such as bias correction, and weight decay. Hyperparameters: Learning rate η , betas = $(\beta_1, \beta_2, \beta_3)$, weight averaging coefficient δ , and epsilon ϵ .

- 1: Sample batch B_t .
- 2: $g \leftarrow -\nabla_w \phi_{B_t}(w_t)$
- 3: $v \leftarrow \beta_2 v + (1 - \beta_2)(g \odot g)$
- 4: $N \leftarrow \frac{\beta_3 m + (1 - \beta_3)g}{\sqrt{\hat{v} + \epsilon}}$
- 5: $w \leftarrow w - \eta N$
- 6: $m \leftarrow \beta_1 m + (1 - \beta_1)g$
- 7: $c = \max(1 - 1/t, 1 - 1/(\delta t))$
- 8: $w_{\text{avg}} \leftarrow c w_{\text{avg}} + (1 - c)w$

2. Standard AdamW with weight averaging
3. Schedule-Free AdamW
4. Accelerated AdamW with weight averaging (Algorithm 3)
5. MARS
6. AdEMAMix
7. Simplified-AdEMAMix

Details of hyperparameter sweeps for these algorithms are provided in Section A.

Figure 1 shows a stratified, easy-to-read ordering that lines up with the theory at every level. At the finest level, Schedule-Free AdamW and accelerated AdamW with tailed weight averaging are effectively indistinguishable in validation loss—the near-coincidence our equivalence result predicts. One step up, both of these accelerated methods

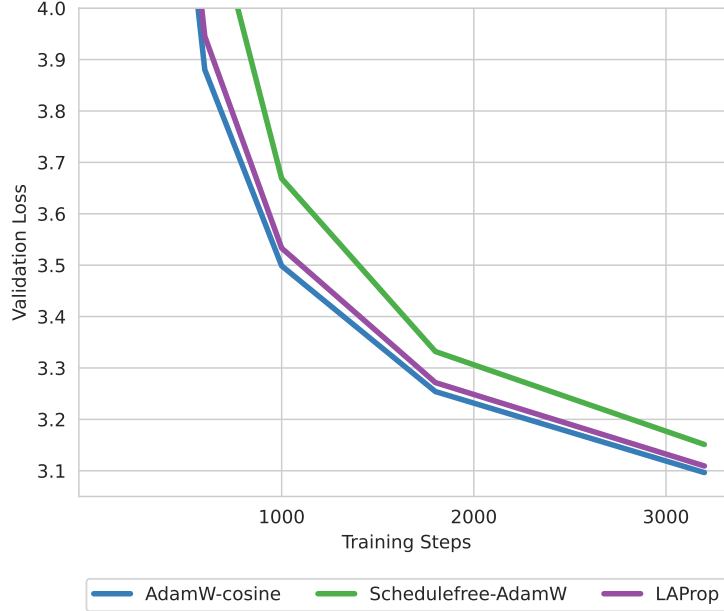


Figure 2: Large-batch regime: best-run validation loss for AdamW with cosine decay, Schedule-Free AdamW, and LProp. The ranking reverses relative to the small-batch setting—Schedule-Free AdamW now trails AdamW with cosine decay by a clear margin, while LProp stays close to AdamW, showing that the momentum/weight-averaging coupling (not the momentum-vs-preconditioning ordering) is what costs Schedule-Free AdamW here. Experimental details can be found in Section 5.1.

improve on the two standard baselines, AdamW with cosine decay and AdamW with tailed weight averaging. And at the top of the ranking, AdEMAMix and Simplified-AdEMAMix achieve the lowest loss of any method in the comparison, consistent with their being the closest structural match to accelerated SGD. The pattern is monotone in “how accelerated” each method is, which is exactly the signal the theory anticipates in a noise-dominated regime.

5.1 LARGE BATCH SIZE EXPERIMENTS

The small-batch results establish the advantage of the accelerated methods; the natural next question is whether that advantage survives outside the noise-dominated regime. To test this we hold the model and dataset fixed and move to the opposite extreme—a batch size of 1 million tokens, training over 3 billion tokens ($\approx$ Chinchilla scale)—and re-run the comparison. The theory predicts the advantage should shrink, and the measurements bear this out on both fronts below.

Schedule-Free AdamW: The direction of the comparison flips relative to the small-batch case. As shown in Figure 2, Schedule-Free AdamW now falls clearly behind AdamW rather than matching an accelerated baseline. We attribute this gap to the coupling between the weight-averaging and momentum coefficients: at large batch size the optimal momentum is well below $1 - 1/t$, and although one could use a scaling factor $\approx 1 - r/t$ for some $r \geq 1$, a larger r shrinks the effective weight-averaging window, so no single setting recovers AdamW’s performance.

To rule out an alternative explanation, we also isolate the effect of update ordering. AdamW applies momentum before preconditioning, whereas Schedule-Free AdamW applies preconditioning before momentum, making it algorithmically similar to LProp (Ziyin et al., 2021). If ordering were the culprit, LProp should also lag AdamW—but Figure 2 shows LProp performing on par with AdamW. This control pins the gap on the momentum/weight-averaging coupling rather than on the ordering of momentum and preconditioning.

AdEMAMix: The large-batch curves in Figure 3 tell a two-part story. First, and consistent with Pagliardini et al. (2024), setting $\beta_1 = 0.0$ in AdEMAMix incurs a clear loss penalty relative to keeping two separate momentum terms.

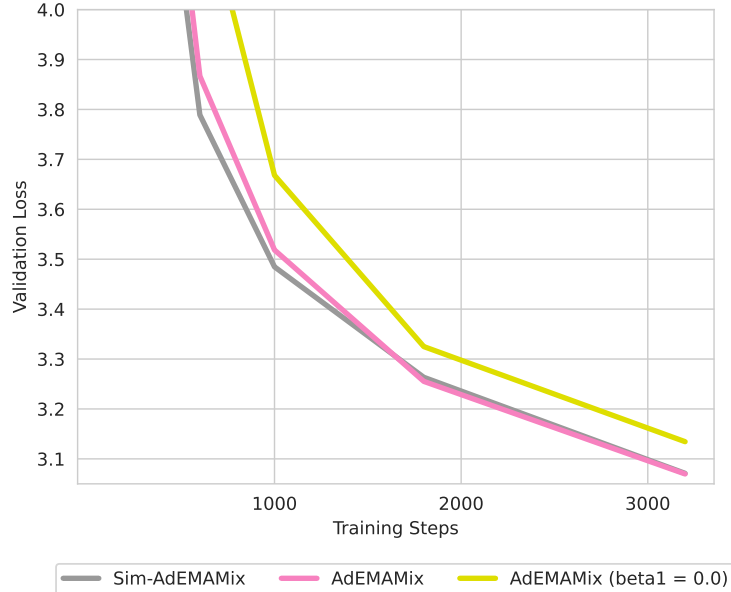


Figure 3: Large-batch regime: best-run validation loss for AdEMAMix (with $\beta_1 = 0.0$ and with two momentum terms) and our Simplified-AdEMAMix. Two comparisons stand out: dropping to $\beta_1 = 0.0$ costs AdEMAMix a visible amount of loss, whereas Simplified-AdEMAMix—using a single momentum buffer—overlaps the full two-buffer AdEMAMix, matching it despite the reduced state. Experimental details can be found in Section 5.1.

This is the predicted failure mode: AdEMAMix pins a fixed weight of 1 on the current gradient, whereas accelerated SGD theory (Gupta et al., 2023) requires that weight to diminish as batch size grows.

Second, and more importantly for practice, Simplified-AdEMAMix closes that penalty entirely. As Figure 3 shows, its validation loss coincides with that of the full two-buffer AdEMAMix while using only a single momentum term. Notably, it reaches this parity at $\alpha = 0.0$ —that is, the best-performing configuration reduces to standard Adam with momentum scheduling, so the same accuracy is obtained at strictly lower memory cost.

6 CONCLUSION

In this work, we establish explicit connections between accelerated SGD variants and several recently proposed optimizers, including Schedule-Free optimizers, AdEMAMix, MARS, and Lion. Our controlled seven-way comparison turns these connections into measurable predictions and confirms them: in the small-batch, noise-dominated regime the optimizers rank in order of how closely they match accelerated SGD, with AdEMAMix and Simplified-AdEMAMix achieving the lowest validation loss and Schedule-Free AdamW matching its accelerated-SGD counterpart. The same lens correctly predicts the large-batch regime, where the advantage over Adam disappears and Schedule-Free AdamW falls behind AdamW with cosine decay owing to its coupled momentum and weight-averaging coefficients.

Building on this connection, we introduce Simplified-AdEMAMix, which removes the need for maintaining two separate momentum buffers. Across both the small- and large-batch regimes it delivers the same validation loss as full AdEMAMix while storing a single momentum buffer instead of two—accuracy held constant at a strictly lower memory cost.

REFERENCES

Chen, X., Liang, C., Huang, D., Real, E., Wang, K., Pham, H., Dong, X., Luong, T., Hsieh, C., Lu, Y., and Le, Q. V. Symbolic discovery of optimization algorithms. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems 36: An-*

- nual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023, 2023.* URL http://papers.nips.cc/paper_files/paper/2023/hash/9a39b4925e35cf447ccba8757137d84f-Abstract-Conference.html.
- Defazio, A. Momentum via primal averaging: Theoretical insights and learning rate schedules for non-convex optimization, 2021. URL <https://arxiv.org/abs/2010.00406>.
- Defazio, A., Yang, X., Mehta, H., Mishchenko, K., Khaled, A., and Cutkosky, A. The road less scheduled. *CoRR*, abs/2405.15682, 2024. doi: 10.48550/ARXIV.2405.15682. URL <https://doi.org/10.48550/arXiv.2405.15682>.
- Gupta, K., Siegel, J., and Wojtowysch, S. Achieving acceleration despite very noisy gradients, 2023. URL <https://arxiv.org/abs/2302.05515>.
- Jain, P., Kakade, S. M., Kidambi, R., Netrapalli, P., and Sidford, A. Accelerating stochastic gradient descent for least squares regression. In Bubeck, S., Perchet, V., and Rigollet, P. (eds.), *Proceedings of the 31st Conference On Learning Theory*, volume 75 of *Proceedings of Machine Learning Research*, pp. 545–604. PMLR, 06–09 Jul 2018. URL <https://proceedings.mlr.press/v75/jain18a.html>.
- Kidambi, R., Netrapalli, P., Jain, P., and Kakade, S. M. On the insufficiency of existing momentum schemes for stochastic optimization. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=rJTutzbA->.
- Liu, C. and Belkin, M. Accelerating sgd with momentum for over-parameterized learning. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=rlgixp4FPH>.
- Lucas, J., Sun, S., Zemel, R., and Grosse, R. Aggregated momentum: Stability through passive damping. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Syxt5oC5YQ>.
- Ma, J. and Yarats, D. Quasi-hyperbolic momentum and adam for deep learning. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=SlfUpoR5FQ>.
- Pagliardini, M., Ablin, P., and Grangier, D. The ademamix optimizer: Better, faster, older. 2024. URL <https://arxiv.org/abs/2409.03137>.
- Porian, T., Wortsman, M., Jitsev, J., Schmidt, L., and Carmon, Y. Resolving discrepancies in compute-optimal scaling of language models. *CoRR*, abs/2406.19146, 2024. doi: 10.48550/ARXIV.2406.19146. URL <https://doi.org/10.48550/arXiv.2406.19146>.
- Vaswani, S., Bach, F., and Schmidt, M. Fast and faster convergence of sgd for over-parameterized models and an accelerated perceptron. In Chaudhuri, K. and Sugiyama, M. (eds.), *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pp. 1195–1204. PMLR, 16–18 Apr 2019. URL <https://proceedings.mlr.press/v89/vaswani19a.html>.
- Yuan, H., Liu, Y., Wu, S., Zhou, X., and Gu, Q. Mars: Unleashing the power of variance reduction for training large models, 2024. URL <https://arxiv.org/abs/2411.10438>.
- Ziyin, L., Wang, Z. T., and Ueda, M. Laprop: Separating momentum and adaptivity in adam, 2021. URL <https://arxiv.org/abs/2002.04839>.

A HYPERPARAMETERS

For a fair comparison, every optimizer in Figure 1 was tuned over its own grid and reported at its best configuration; the small-batch sweeps are listed below.

1. AdamW with cosine decay - 51.2k warmup - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.99, 0.999, 0.99968, 0.9999]$. The optimal values of β_1 and β_2 were .9 and .999 respectively matching the default values. We note that for larger batch sizes it is common to use $\beta_2 = .95$, the benefit of higher β_2 at smaller batch sizes has also been observed by Porian et al. (2024).
2. AdamW with cosine decay - 10k warmup - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ i.e. we fix β_1, β_2 to be the optimal values from the previous sweep. This performed worse than warmup of 51.2k steps.
3. AdamW constant fraction weight averaging: - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, β_2 in $[0.99, 0.997, 0.999, 0.9997]$, δ in $[0.05, 0.1, 0.2]$.
4. AdamW with cosine decay and weight averaging - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, δ in $[0.025, 0.05, 0.1]$.
5. Accelerated SGD based AdamW with cosine decay - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.999, 0.99968, 0.9999]$, β_2 in $[0.99, 0.9968, 0.999]$, $\beta_3 = 0.9$
6. Accelerated SGD based AdamW with constant learning rate and weight averaging - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.99684, 0.999]$, β_2 in $[0.999]$, $\beta_3 = 0.9$, δ in $[0.05, 0.1]$
7. Accelerated SGD based AdamW with cosine decay and weight average - learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.99684, 0.999]$, $\beta_2 = 0.999$, δ in $[0.05, 0.1]$, $\beta_3 = 0.9$
8. Schedulefree AdamW with constant learning rate - learning rate in $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.8, 0.9, 0.95]$, $\beta_2 = 0.999$
9. Schedulefree AdamW with cosine decay - $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.8, 0.9, 0.95]$, $\beta_2 = 0.999$
10. MARS - $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95, 0.99]$, β_2 in $[0.99, 0.999]$, γ in $[0.0, 0.01, 0.02, 0.03, 0.04, 0.05]$, precondition 1d was set to True.
11. AdEMAMix - $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.999$, β_3 in $[0.99, 0.999, 0.9999]$, α in $[2, 4, 8, 16]$.
12. Sim-AdEMAMix - $[1e-6, 3.16e-6, 1e-5, 3.16e-5]$, β_1 in $[0.99, 0.999, 0.9999]$, $\beta_2 = 0.999$, α in $[10, 20, 50, 100]$

The corresponding best-of-sweep configurations underlying the large-batch comparisons are provided below:

1. Schedule-Free AdamW: $[1e-3, 3.16e-3, 1e-2]$, β in $[0.8, 0.9, 0.95]$, β_2 in $[0.9, 0.95]$, r in $[0.0, 5.0, 9.0, 50.0]$
2. AdamW: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.9, 0.95]$
3. LAMProp: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95]$, β_2 in $[0.9, 0.95]$
4. AdEMAMix: $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.95$, β_3 in $[0.9, 0.95, 0.99]$, α in $[2, 4, 8, 16]$
5. Sim-AdEMAMix: $[1e-4, 3.16e-4, 1e-3]$, β_1 in $[0.9, 0.95, 0.99]$, $\beta_2 = 0.95$, α in $[0.0, 0.5, 1.0]$

B EQUIVALENCE OF PREVIOUS ACCELERATION METHODS

The general accelerated SGD form is provided in Equation (1). In this section, we will show that all the methods in the works Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023) fall within this form.

B.1 AGNES

The update for Gupta et al. (2023) is given below:

$$x'_n = x_n + \alpha v_n \quad x_{n+1} = x'_n - \eta g'_n \quad v_{n+1} = \rho_n(v_n - g'_n)$$

where g'_n is stochastic gradient evaluated on x'_n and the final function is evaluated on x_n . The above equations can be rewritten as

$$x'_{n+1} = x'_n - \eta g'_n + \alpha v_{n+1} \quad - \frac{v_{n+1}}{\rho_n} = \rho_{n-1} \left(- \frac{v_n}{\rho_{n-1}} \right) + g'_n$$

Thus x'_{n+1} follows update equation of the form of Equation (1).

B.2 ASGD

The update for Jain et al. (2018) is given by:

$$y_{j-1} = \alpha x_{j-1} + (1 - \alpha)v_{j-1} \quad x_j = y_{j-1} - \delta g_{j-1} \quad z_{j-1} = \beta y_{j-1} + (1 - \beta)v_{j-1} \quad v_j = z_{j-1} - \gamma g_{j-1}$$

where g_{j-1} represents the stochastic gradient evaluated on y_{j-1} and the function is evaluated on the tail averaged x .

The update equations above can be rewritten as:

$$y_j = y_{j-1} - \alpha \delta g_{j-1} - (1 - \alpha)[y_{j-1} - v_j] \quad \frac{y_{j-1} - v_j}{\gamma - (1 - \beta)\alpha\delta} = (1 - \beta)\alpha \frac{y_{j-2} - v_{j-1}}{\gamma - (1 - \beta)\alpha\delta} + g_{j-1}$$

The update equations above follow the form of Equation (1).

B.3 MASS

The update for Liu & Belkin (2020) is given by:

$$w_{t+1} = u_t - \eta_1 g_t \quad u_{t+1} = (1 + \gamma)w_{t+1} - \gamma w_t + \eta_2 g_t$$

where g_t is the stochastic gradient evaluated on u_t and the function is evaluated on w_t . These equations can be rewritten as:

$$u_{t+1} = u_t - \gamma(w_t - u_t) - [\eta_1(1 + \gamma) - \eta_2]g_t \quad \frac{w_t - u_t}{\eta_1\gamma - \eta_2} = \gamma \frac{w_{t-1} - u_{t-1}}{\eta_1\gamma - \eta_2} + g_t$$

The update equations above follow the form of Equation (1).

B.4 SGD WITH NESTEROV ACCELERATION

The update for Vaswani et al. (2019) is given by:

$$w_{k+1} = \zeta_k - \eta g_k \quad \zeta_k = \alpha_k v_k + (1 - \alpha_k)w_k \quad v_{k+1} = \beta_k v_k + (1 - \beta_k)\zeta_k - \gamma_k \eta g_k$$

These equations can be rewritten as:

$$\zeta_{k+1} = \zeta_k - \eta g_k + \alpha_{k+1}[v_{k+1} - w_{k+1}]; \quad v_{k+1} - w_{k+1} = \beta_k(1 - \alpha_k)[v_k - w_k] - \eta(\gamma_k - 1)g_k$$